

数据结构合并

在很多时候，我们对多个数据进行合并，如果是用普通的数组来维护，那么直接相同下标累加就可以了，但是如果每次的数据都是用数据结构来维护的，那么又该如何把这两个数据结构进行合并呢？

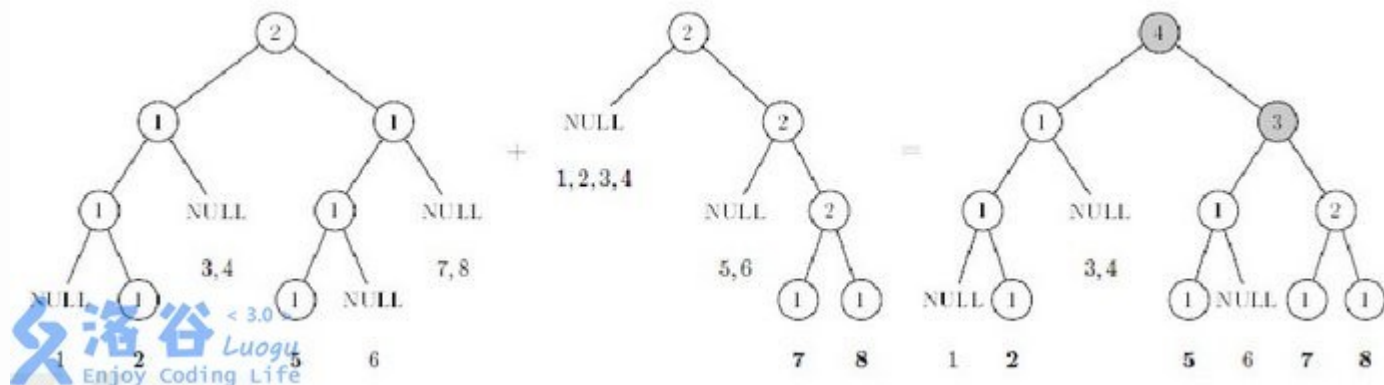
常见的数据结构合并包括并查集，堆(可并堆)，线段树，平衡树等等。这些数据结构其实都是合并的，而且合并并不是直接把其中一个数据结构中的有用节点直接一个一个放入另一个数据结构中，因为这样做时间复杂度太高了。所以需要另一种方法。

并查集的启发式合并以前我们就讲过，就不再讲了，其余的可并堆，线段树，平衡树合并原理大致相同，我们这里以常见的线段树为例。

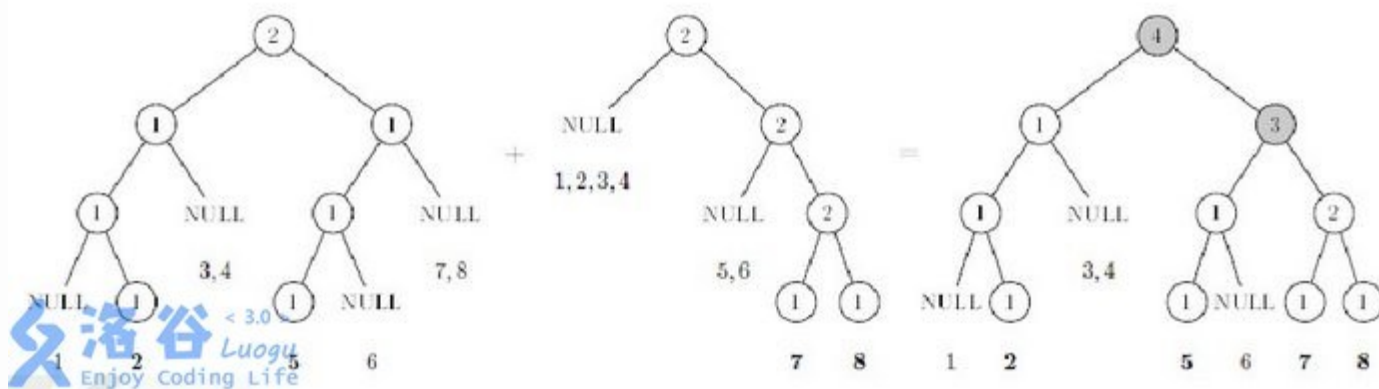
数据结构合并

线段树要合并一般都是权值线段树，也就是每个点记录的是数量信息，包括平衡树也是一样，几乎不存在普通区间线段树合并，没有太大意义。

线段树合并就是建一棵新的线段树保存原有的两颗线段树的信息。

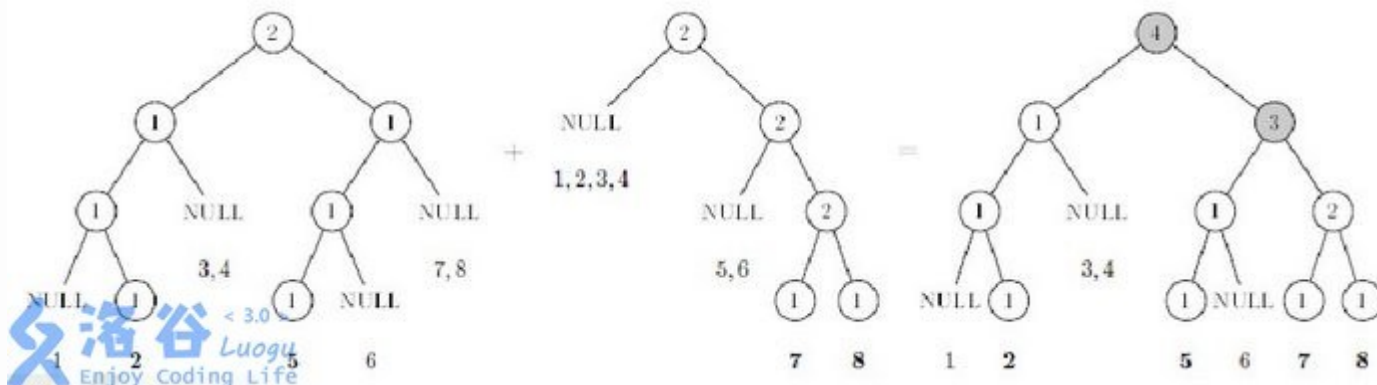


数据结构合并



还是和以前一样，我们仔细观察一下这三颗树之间的区别和联系。我们会发现，对于同一个节点，如果需要合并的两棵树的这个节点都存在，那么该节点信息会发生改变，那么就需要新开节点来保存。对于同一个节点，如果需要合并的两颗树一棵存在该节点，一棵不存在该节点，那么我们会发现直接把有的那个节点接在下面就可以返回了，后面的信息都不用变了。这就是线段树合并的精髓之处。

数据结构合并



所以线段树的合并，实际上就是：

如果a有pos位置，b没有，那么新的线段树pos位置赋成a，返回

如果b有pos位置，a没有，赋成b，返回

如果此时已经合并到两棵线段树的叶子节点了，就把b在pos的值加到a上，把新线段树上的pos位置赋成a，返回

递归处理左子树

递归处理右子树

用左右子树的值更新当前节点

将新线段树上的pos位置赋成a，返回

数据结构合并

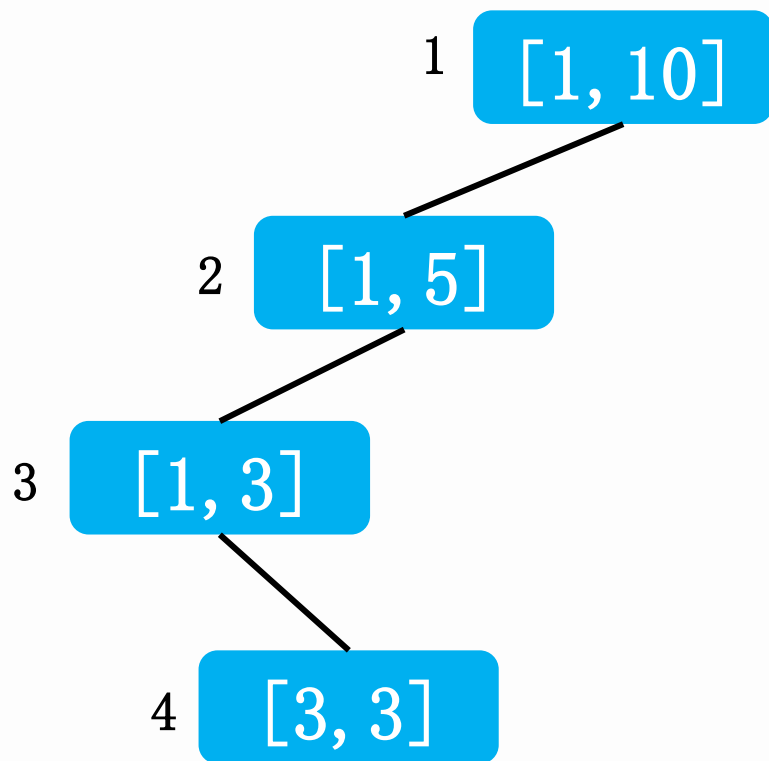
其实我们发现线段树的合并和我们前面学习的可持久化数据结构的实现方式是比较类似的，都是如果可以沿用以前的节点就直接沿用以前的节点，否则就新开节点。

但是这里有一个问题——当有一棵线段树中某个节点为空时就不需要新增，否则就要新增，对于同一颗线段树上的点，树是一开始就初始化好了的，每棵树的形态都一样，就算一棵线段树只有一个有用节点，另一棵线段树也只有一个有用节点，但是树的形态还是完全的。这样对于每一个节点都要重新开，完全没有实际意义。

这就是要用到以前讲过的另一种线段树动态开点的办法了，使用时再开点，用不到时就直接放一个整体区间在那。

这样如果只有一个节点，那么就只有从根节点到该叶子节点这一条链有节点。

数据结构合并





数据结构合并

我们先来看一道比较简单的题：

洛谷 P3521

P012011 ROT-Tree Rotations

给出一棵 n 个叶子的二叉树，仅叶子有点权，求通过交换任意几个点的左右子树使得前序遍历叶子的逆序对最小值。



数据结构合并

我们会发现一个问题，在一个子树内交换左右儿子，对于子树之外的结点没有影响。所以我们可以dfs整颗树，对于一个结点的左右儿子，如果交换后左右儿子各出一个组成的逆序对更少则交换，否则就不交换。

关键是如何判断两棵子树合并过程中交换和不交换位置的逆序对的数量呢？

以前我们求动态逆序对都是树状数组来求解的，当然权值线段树也可以做，我们思考一下这道题权值线段树是否可以做。

数据结构合并

对于相邻两棵子树，局部最优才能得到整体最优，局部不是最优那么整体一定不是最优。

比如一棵子树里面两个结点是5, 3，那么只有这棵子树的逆序对最少了，那么合并之后的树的逆序对才可能更少，所以那么这棵子树最后一定是左儿子为3，右儿子为5。

我们把其中一颗子树看做一个整体，如果这个整体内的数如何变化，那么另一颗子树相对于这个整体的逆序对的数量是不会变化的。所以我们只需要保证整颗子树内部整体逆序对数量最少，那么最后就一定能保证最终答案最优。那么合并之后的逆序对的计算方法就是左子树内部的逆序对+左子树相对于右子树的逆序对+右子树内部的逆序对。



数据结构合并

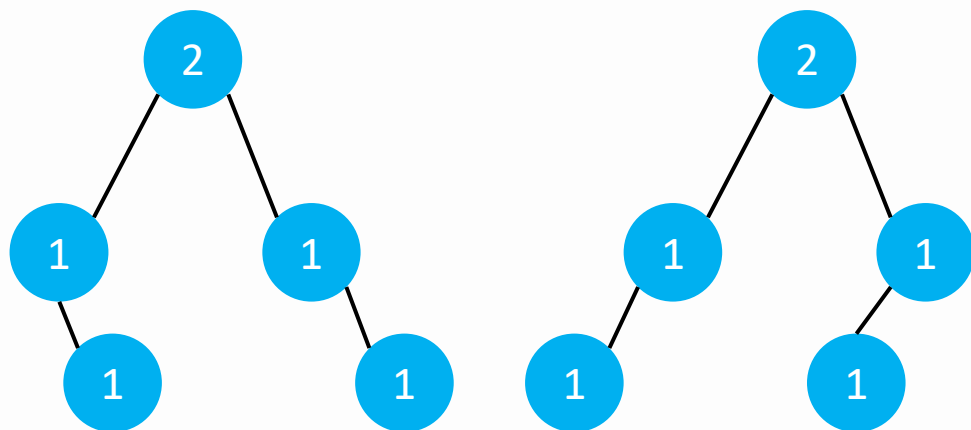
那么如何求解左子树相对于右子树的逆序对呢？

已知两个子区间的逆序对的数量，我们是没有办法直接求出来合并之后的区间的逆序对的数量。

我们重新画图来研究一下，如果对于每一颗子树我都用一棵线段树来维护实际上是可以实现的。



数据结构合并



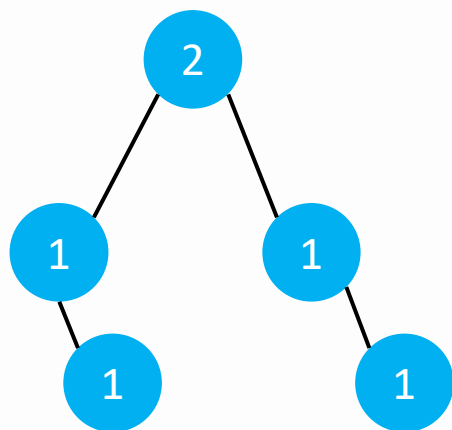
左子树A
树上有值2, 4

右子树B
树上有值1, 3

当我们从根结点开始遍历的时候，我们是可以计算出部分逆序对的，哪一部分呢？ $A.r$ 和 $B.l$ 一定构成逆序对，证明很明显，因为两棵线段树代表的区间是相同的。所以一共有 $A.r.size * B.l.size$ 如果交换，那么就反过来

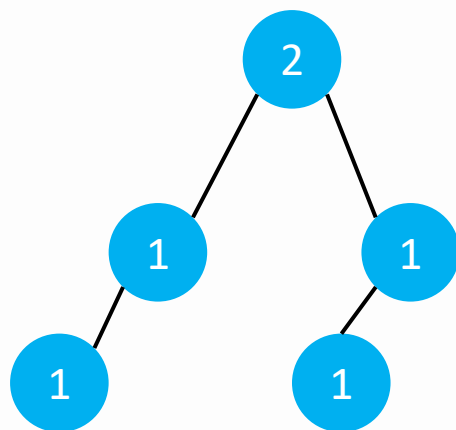
第一层算出来
不交换有逆序对1对
交换有逆序对1对

数据结构合并



左子树A
树上有值2, 4

第二层左子树算出来
不交换有逆序对1对
交换有逆序对0对

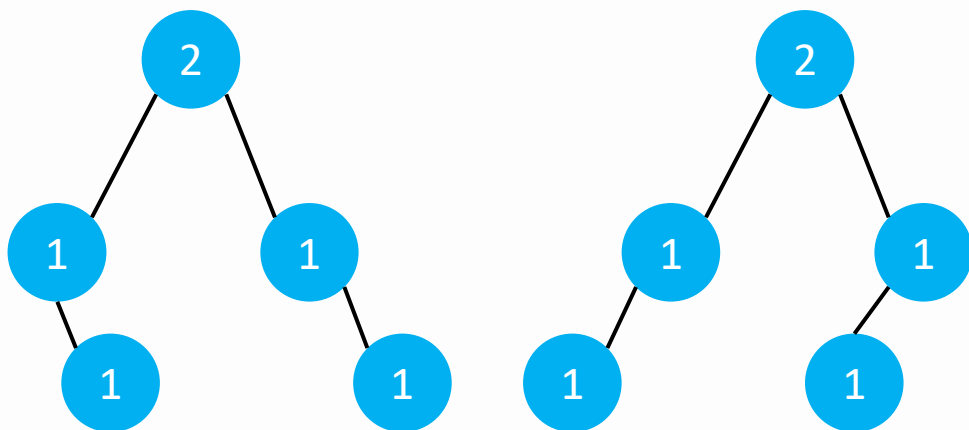


右子树B
树上有值1, 3

第二层右子树算出来
不交换有逆序对1对
交换有逆序对0对

当我们遍历到第二层的时候，对于左子树，同样还是用A树右儿子的size*B树左儿子的size来表示不交换的逆序对，如果交换，那么就是B数右儿子的size*A树左儿子的size.

数据结构合并



左子树A

树上有值2, 4

右子树B

树上有值1, 3

第三层左子树算出来

交换和不交换逆序对数量都是0. 所以总体来说不交换逆序对为3, 交换逆序对1, 所以需要交换左右子树

数据结构合并

```
int merge(int x,int y,int l,int r)
{
    if(!x||!y)
        return x+y;
    if(l==r)
    {
        tree[x].size+=tree[y].size;
        return x;//把q合并到p上
    }
    u+=tree[tree[x].r].size*tree[tree[y].l].size;
    v+=tree[tree[y].r].size*tree[tree[x].l].size;
    int mid=(l+r)/2;
    tree[x].l=merge(tree[x].l,tree[y].l,l,mid);
    tree[x].r=merge(tree[x].r,tree[y].r,mid+1,r);
    tree[x].size=tree[tree[x].l].size+tree[tree[x].r].size;
    return x;
}
```



数据结构合并

BZOJ 4756

给出一个 n 个点的带点权的以1为根的树，求出每个点的子树中有多少个权值比该点大的点。



数据结构合并

这道题就是一道很明显的线段树合并，因为需要合并子树，而且直接合并得不到我们需要答案，所以我们考虑线段树合并。

我们对于每一个点 u ，建立一颗线段树保存子树 u 中的所有权值，那么 $ans[u]$ 就等于线段树中比 $val[u]$ 大的值有多少，这在一颗线段树中就非常好求的。

那么关键是子树如何合并呢？

只需要把 u 的所有子树的线段树直接合并起来就可以了
总体时间复杂度只有 $n \log n$ 。



数据结构合并

第一步：

因为数的范围很大，所以在用权值线段树之前先离散化

第二步：

对每一个结点建一颗线段树



数据结构合并

```
void build(int l,int r,int k)
{
    int now=++k;
    if(l==r)
    {
        tree[now].size=1;
        return now;
    }
    int mid=(l+r)/2;
    if(k<=mid)//动态建树,不存在的点就不要了
        tree[now].l=build(l,mid,k);
    else
        tree[now].r=build(mid+1,r,k);
    pushup(now);//更新size
    return now;//返回根结点
}
for(int i=1;i<=n;i++)
    root[i]=build(1,n,a[i]);
```

数据结构合并

第三步：

从根结点开始递归遍历这棵树，在返回的时候对多颗子树进行线段树合并，最后遍历结束得到答案。

```
int merge(int x,int y)//新不新开结点根据题目需要
{ //线段树合并
    if(!x||!y)
        return x+y;
    tree[x].l=merge(tree[x].l,tree[y].l);
    tree[x].r=merge(tree[x].r,tree[y].r);
    pushup(x);
    return x;
}
```

数据结构合并

```
int query(int id,int L,int R,int l,int r)
{//线段树区间查询
    if(l==L&&r==R)
    {
        return tree[id].size;
    }
    int mid=(L+R)/2;
    if(r<=mid)
        return query(tree[id].l,L,mid,l,r);
    else if(l>mid)
        return query(tree[id].r,mid+1,R,l,r);
    else
    {
        return query(tree[id].l,L,mid,l,mid)+query(tree[id].r,mid+1,R,mid+1,r);
    }
}
```

数据结构合并

```
void dfs(int u,int fa)
{
    for(int i=head[u];i;i=edge[i].next)
    {
        int v=edge[i].to;
        if(v!=fa)
            dfs(v,u);
        root[u]=merge(root[u],root[v]);//合并
    }
    ans[u]=query(root[u],1,n,a[u]+1,n);
    //询问区间后半段有多少符合条件的数
}
```

数据结构合并

题目描述

首先村落里的一共有 n 座房屋，并形成一棵树状结构。然后救济粮分 m 次发放，每次选择两个房屋 (x, y) ，然后对于 x 到 y 的路径上(含 x 和 y)每座房子里发放一袋 z 类型的救济粮。

然后深绘里想知道，当所有的救济粮发放完毕后，每座房子里存放的最多的是哪种救济粮。

输入格式

输入的第一行是两个用空格隔开的正整数，分别代表房屋的个数 n 和救济粮发放的次数 m 。

第 2 到第 n 行，每行有两个用空格隔开的整数 a, b ，代表存在一条连接房屋 a 和 b 的边。

第 $(n + 1)$ 到第 $(n + m)$ 行，每行有三个用空格隔开的整数 x, y, z ，代表一次救济粮的发放是从 x 到 y 路径上的每栋房子发放了一袋 z 类型的救济粮。

输出格式

输出 n 行，每行一个整数，第 i 行的整数代表 i 号房屋存放最多的救济粮的种类，如果有多种救济粮都是存放最多的，输出种类编号最小的一种。

如果某座房屋没有救济粮，则输出 0。

数据结构合并

拿到题一看，第一眼就觉得是树剖，剖完之后用线段树维护，但是发现线段树维护不了，也不是维护不了，应该是直接维护不可以维护。

对于树上路径修改问题，不但可以树剖，还可以树上差分，特别是这种多次修改只有一次询问这种，一般树上差分就是这种套路。

差分之后，我们可以对于每一个结点数量最多的救济粮的型号，对于每一个子树的根结点，他的数量最多的救济粮的幸好就等于他的子树中最多的救济粮型号，又是子树问题，直接用线段树合并就可以做。

每个结点的答案得到了，要求一条链的最大值，可以直接再树剖或者树上倍增就可以了。



数据结构合并

看了前面几道题，我们会发现线段树合并主要是用来处理树上的子树合并问题，如果一棵子树的根结点等于其所有子树结点信息的合并，但是直接合并要么效率很低，要么不可以直接dfs遍历就可以得到，那么可以考虑下线段树合并，对于每一个结点都开一棵线段树来维护。





练习题

洛谷	4556	雨天的尾巴
洛谷	4219	大融合
洛谷	3224	永无乡

